🚀 Space Shooter - Complete Unity Build Instructions

Table of Contents

Prerequisites

Required Software

- **Unity Hub** (latest version): https://unity.com/download
- **Unity Editor 2021.3 LTS or newer** (2022.x or 2023.x also work)
- When installing via Unity Hub, ensure the **Windows Build Support** module is included
- **Visual Studio 2019/2022** (Community edition is free) or **VS Code** with C# extension

System Requirements

- Windows 10/11 (64-bit)
- 8 GB RAM minimum
- 10 GB free disk space (for Unity installation)

Project Setup

Step 1: Create a New Unity Project

1. Open **Unity Hub**
2. Click **“New Project”**
3. Select the **“2D (Built-in Render Pipeline)”** template
   - ⚠️ **IMPORTANT**: Must be 2D, not 3D or URP
4. Set the **Project Name** to `SpaceShooter`
5. Choose your preferred **Location** on disk
6. Click **“Create project”**
7. Wait for Unity to initialize the project (may take 1-2 minutes)

Step 2: Import All Scripts

1. In the Unity Editor, locate the **Project** panel (bottom of screen by default)
2. Right-click on the **Assets** folder → **Show in Explorer**
3. This opens the `Assets` folder in Windows Explorer
4. **Copy the following folders** from this repository into the Assets folder:

```
Assets/
  ├── Scripts/          ← Copy this entire folder
  │   ├── Core/
  │   │   ├── GameManager.cs
  │   │   ├── HealthSystem.cs
  │   │   └── CollisionHandler.cs
  │   ├── Player/
  │   │   └── PlayerController.cs
  │   ├── Enemies/
  │   │   ├── EnemyController.cs
  │   │   └── EnemySpawner.cs
  │   ├── Weapons/
  │   │   └── BulletController.cs
  │   ├── PowerUps/
  │   │   └── PowerUpController.cs
  │   ├── UI/
  │   │   ├── UIManager.cs
  │   │   └── MainMenuUI.cs
  │   ├── Background/
  │   │   └── BackgroundScroller.cs
  │   ├── Audio/
  │   │   └── AudioManager.cs
  │   └── Utilities/
  │       ├── SpriteGenerator.cs
  │       ├── GameSetup.cs
  │       └── MainMenuSetup.cs
```

5. Return to Unity Editor and wait for it to compile scripts (progress bar at bottom)
6. Check the **Console** panel (Window → General → Console) for any errors
   - There should be **zero errors** if all scripts are copied correctly

Step 3: (Optional) Copy Project Settings

You can optionally copy the `ProjectSettings/` files from this repository to pre-configure tags, layers, and physics. If you prefer to set them up manually, follow the Tag & Layer Setup section below.

Tag & Layer Setup

Required Tags

These tags are essential for the collision system to work properly.

1. Go to **Edit → Project Settings → Tags and Layers**
2. Expand the **Tags** section

3. Click **+** to add these custom tags:
   - `PlayerBullet`
   - `EnemyBullet`
   - `PowerUp`

> **Note:** The `Player` and `Enemy` tags are built-in Unity tags and should already exist. The `Untagged` tag is the default.

Verify Built-in Tags Exist

Make sure these exist (they should by default):
- `Player`
- `Enemy` (may need to be added if not present)

Layers (Optional but Recommended)

For future physics layer optimization, you can add:
- Layer 8: `Player`
- Layer 9: `Enemy`
- Layer 10: `PlayerBullet`
- Layer 11: `EnemyBullet`
- Layer 12: `PowerUp`

Scene Configuration

Step 4: Create the Main Menu Scene

1. Go to **File → New Scene** (choose “Basic 2D” if prompted)
2. **Delete** any default GameObjects in the Hierarchy (except the Main Camera)
3. Select the **Main Camera** in Hierarchy:
   - Set **Background** color to `#050510` (very dark blue-black)
   - Ensure **Projection** is set to **Orthographic**
   - Set **Size** to `5`
4. Create an empty GameObject:
   - **Right-click** in Hierarchy → **Create Empty**
   - Name it `MainMenuSetup`
   - In the Inspector, click **Add Component** → search for `MainMenuSetup` → add it
5. Save the scene: **File → Save As** → navigate to `Assets/Scenes/` → name it `MainMenu.unity`

Step 5: Create the Game Scene

1. Go to **File → New Scene** (choose “Basic 2D” if prompted)
2. **Delete** any default GameObjects (except Main Camera)
3. Select the **Main Camera**:
   - Set **Background** color to `#050510` (very dark blue-black)
   - Ensure **Projection** is set to **Orthographic**
   - Set **Size** to `5`
4. Create an empty GameObject:
   - **Right-click** in Hierarchy → **Create Empty**
   - Name it `GameSetup`
   - In the Inspector, click **Add Component** → search for `GameSetup` → add it

5. Save the scene: **File → Save As** → navigate to `Assets/Scenes/` → name it `GameScene.unity`

Step 6: Configure Build Settings

1. Go to **File → Build Settings**
2. Click **“Add Open Scenes”** (or drag scenes from Project panel)
3. Ensure the scenes are in this order:
   `0: Scenes/MainMenu` ← This MUST be index 0 (loaded first)
   `1: Scenes/GameScene`
   - You can drag scenes to reorder them
4. Ensure **Platform** is set to **Windows, Mac, Linux** (or “PC, Mac & Linux Standalone”)
   - If not selected, click on it and click **“Switch Platform”**
5. Close the Build Settings window

Building the Game

Step 7: Build as Windows Executable

1. Open **File → Build Settings**
2. Verify:
   - Platform: **PC, Mac & Linux Standalone**
   - Target Platform: **Windows**
   - Architecture: **x86_64** (recommended) or **x86**
3. Click **“Player Settings…”** (bottom-left) and configure:
   - **Product Name**: `Space Shooter`
   - **Company Name**: (your name or company)
   - **Default Screen Width**: `1920`
   - **Default Screen Height**: `1080`
   - **Fullscreen Mode**: `Fullscreen Window` or `Windowed`
   - Under **Other Settings**:
     - **Color Space**: `Gamma` (for 2D games)
     - **Api Compatibility Level**: `.NET Standard 2.1` or `.NET Framework`
4. Close Player Settings
5. Click **“Build”** (or **“Build and Run”**)
6. Choose a folder for the build output (e.g., `Builds/Windows/`)
7. Wait for the build to complete (1-3 minutes)

Build Output

Your build folder will contain:

```
Builds/Windows/
├── Space Shooter.exe          ← The game executable
├── Space Shooter_Data/        ← Game data (must stay next to .exe)
├── UnityCrashHandler64.exe    ← Crash reporter
└── UnityPlayer.dll            ← Unity runtime
```

Distribution

To distribute the game:
1. **Zip** the entire build folder
2. The recipient extracts and runs `Space Shooter.exe`
3. No Unity installation needed on the player's machine

Testing in the Editor

Quick Play Test

1. Open the **MainMenu** scene (`Assets/Scenes/MainMenu.unity`)
2. Press the **Play** button (▶) at the top of the Editor
3. You should see:
   - A starfield background
   - "SPACE SHOOTER" title text
   - "START GAME" and "QUIT" buttons
   - Control instructions
4. Click **"START GAME"** to load the GameScene
5. Test the controls (see below)

Expected Behavior

- Player ship appears at bottom center (cyan triangle)
- Enemies spawn from the top in waves
- Press SPACE to shoot
- Collect power-ups (colored hexagons)
- Score increases when enemies are destroyed
- Health decreases when hit
- Game Over when health reaches 0
- Pause with ESC

Game Controls

Key	Action
W / ↑	Move Up
S / ↓	Move Down
A / ←	Move Left
D / →	Move Right
SPACE	Shoot
ESC	Pause / Resume

Architecture Overview

Script Dependency Map

```
GameManager (Singleton) ← Central hub, manages state/score/waves
    ↑
    ├── PlayerController → HealthSystem, BulletController, CollisionHandler
    ├── EnemyController → HealthSystem, BulletController, CollisionHandler
    ├── EnemySpawner → Creates enemies, manages waves
    ├── PowerUpController → Applied to player on pickup
    ├── UIManager → Displays HUD, pause, game over
    ├── MainMenuUI → Main menu buttons
    ├── AudioManager (Singleton) ← Sound effects
    └── BackgroundScroller → Parallax star background

Utilities:
    ├── SpriteGenerator → Creates all sprites procedurally (no image files needed!)
    ├── GameSetup → Bootstraps the entire GameScene at runtime
    └── MainMenuSetup → Bootstraps the MainMenu scene at runtime
```

Key Design Decisions

1. **Runtime Procedural Generation**: All sprites, UI, and game objects are created programmatically via `GameSetup.cs` and `MainMenuSetup.cs`. No manual Inspector setup is required beyond adding these bootstrap scripts to empty GameObjects.
2. **Singleton Pattern**: `GameManager` and `AudioManager` use DontDestroyOnLoad singletons so they persist across scene transitions.
3. **Event-Driven Architecture**: GameManager fires C# events (`OnScoreChanged` , `OnWaveChanged` , etc.) that other scripts subscribe to. This keeps coupling loose.
4. **Component-Based Health**: `HealthSystem` is a generic component used by both the player and enemies, following Unity's component architecture.
5. **Tag-Based Collisions**: `CollisionHandler` uses Unity tags to determine what collided with what, keeping collision logic centralized.

Enemy Types

Type	Color	Health	Speed	Move-ment	Shooting	Score
Basic	Red	30	3	Straight Down	Single For-ward	100
Fast	Yellow	20	5	Zigzag	Aimed at Player	150
Tank	Purple	60	2	Sine Wave	3-Way Spread	250

Power-Up Types

Type	Color	Effect
Health	Green	Restores 30 HP
Weapon	Orange	Upgrades weapon (up to level 3)
Shield	Blue	5 seconds of invincibility
Speed	Yellow	1.5x speed for 5 seconds

Weapon Levels

Level	Pattern	Fire Rate
1	Single shot	0.20s
2	Double parallel	0.18s
3	Triple spread	0.16s

Troubleshooting

Common Issues

“Tag ‘PlayerBullet’ is not defined”

- Go to Edit → Project Settings → Tags and Layers
- Add the missing tags: `PlayerBullet`, `EnemyBullet`, `PowerUp`

“Scene ‘GameScene’ couldn’t be loaded”

- Open File → Build Settings
- Ensure both `MainMenu` and `GameScene` are added to the build
- MainMenu must be at index 0

Scripts won’t compile / CS errors

- Ensure you’re using Unity 2021.3+
- Check Edit → Project Settings → Player → Api Compatibility Level is `.NET Standard 2.1`
- Make sure ALL script files were copied (check the Scripts folder structure)

Player doesn’t move

- Check that GameManager state is “Playing”
- Verify the Player object has the tag “Player”
- Check that Rigidbody2D has gravity set to 0

Enemies don’t spawn

- Ensure the GameSetup object exists in the GameScene

- Check Console for warnings about missing prefabs
- Verify GameManager.CurrentState is “Playing”

Buttons don’t work

- Ensure an EventSystem exists in the scene
- The bootstrap scripts create one automatically, but verify in Hierarchy

No sound effects

- This is expected! The AudioManager is set up with integration points.
- To add sounds: assign AudioClip assets to the AudioManager’s soundEffects array
- Expected clip names: “PlayerShoot”, “EnemyShoot”, “Explosion”, “PowerUp”

`linearVelocity` error (Unity < 2023)

- In `PlayerController.cs`, change `_rb.linearVelocity` to `_rb.velocity`
- This API was renamed in Unity 2023+

Adding Sound Effects (Optional)

1. Import `.wav` or `.ogg` audio files into `Assets/Audio/SFX/`
2. Select the **AudioManager** GameObject (or find it in DontDestroyOnLoad)
3. In the Inspector, expand **Sound Effects** array
4. Add entries with these names:
   - `PlayerShoot` - Short laser/pew sound
   - `EnemyShoot` - Different laser sound
   - `Explosion` - Explosion boom
   - `PowerUp` - Pickup chime
5. Drag the audio clips to the corresponding entries

Free Sound Effect Resources

- https://freesound.org
- https://opengameart.org
- https://kenney.nl/assets (search for “audio”)

Adding Custom Sprites (Optional)

The game generates all sprites procedurally, but you can replace them:

1. Import `.png` sprite files into the appropriate `Assets/Sprites/` subfolder
2. Select the imported sprite in the Project panel
3. In the Inspector:
   - **Texture Type**: Sprite (2D and UI)
   - **Pixels Per Unit**: 32 (or match your sprite size)
   - Click **Apply**
4. To use custom sprites, you’ll need to modify the `GameSetup.cs` to load your sprites instead of generating them procedurally.

Project File Summary

```
space_shooter_game/
├── Assets/
│   ├── Scripts/
│   │   ├── Core/
│   │   │   ├── GameManager.cs          # Central game state & wave management
│   │   │   ├── HealthSystem.cs         # Reusable HP component
│   │   │   └── CollisionHandler.cs     # All collision logic
│   │   ├── Player/
│   │   │   └── PlayerController.cs     # Movement, shooting, power-ups
│   │   ├── Enemies/
│   │   │   ├── EnemyController.cs      # 5 movement + 4 shoot patterns
│   │   │   └── EnemySpawner.cs         # Wave-based spawning
│   │   ├── Weapons/
│   │   │   └── BulletController.cs     # Generic bullet behavior
│   │   ├── PowerUps/
│   │   │   └── PowerUpController.cs    # 4 power-up types
│   │   ├── UI/
│   │   │   ├── UIManager.cs            # HUD, pause, game over
│   │   │   └── MainMenuUI.cs           # Main menu logic
│   │   ├── Background/
│   │   │   └── BackgroundScroller.cs   # Parallax scrolling
│   │   ├── Audio/
│   │   │   └── AudioManager.cs         # Sound system (singleton)
│   │   └── Utilities/
│   │       ├── SpriteGenerator.cs      # Procedural sprite creation
│   │       ├── GameSetup.cs            # GameScene bootstrapper
│   │       └── MainMenuSetup.cs        # MainMenu bootstrapper
│   ├── Scenes/          # Create MainMenu.unity and GameScene.unity here
│   ├── Prefabs/         # (Generated at runtime - folder for future use)
│   ├── Sprites/         # (Generated at runtime - folder for custom sprites)
│   ├── Audio/           # Place .wav/.ogg files here
│   └── UI/              # UI assets folder
├── ProjectSettings/
│   ├── ProjectSettings.asset
│   ├── TagManager.asset
│   ├── Physics2DSettings.asset
│   └── EditorBuildSettings.asset
└── BUILD_INSTRUCTIONS.md   # This file
```

Happy shooting! 🎮🚀